Course Title	**High Performance Computing (HPC)**		
Course Code	CS301	Credit Structure	3-0-3-4.5 (L-T-P-C)
Category	Core	Semester	Winter Semester (AY 25-26)
Program	Sem VI students of B.Tech (hons in ICT with minor in CS)		
Prerequisites	Understanding of algorithms and data structures. Good background in C programming and Linux. Basic understanding of Computer Architecture.		
Course Objectives/ Brief Course Description	This course aims to provide a systems perspective towards achieving the maximum possible performance out of a particular computing system for a particular algorithm/ problem. This course is an introduction to parallel computing and aims at teaching basic models of parallel programming including the principles of parallel algorithm design, modern processor and parallel computer architectures, performance considerations, programming models for shared and distributed-memory systems, message passing programming models used for cluster computing etc. as well as design of some important scientific and engineering algorithms for parallel systems. Important aspects of OpenMP and MPI programming are covered in the course, and lab-assignments are designed accordingly focussed on implementation of important parallel kernels on shared and distributed memory systems.		
Evaluation/ Grading Policy	• **3 Exams:** Two In-sems and Final End-Sem examination • **8-9 Lab Assignments:** A major part of the course includes Lab Component for actual implementation after learning the basics of Parallel programming and HPC. Grading scheme is relative and depends on both: class performance and minimum expectation from a student. Mid-Sem-1 - 15%; Mid-Sem-2 - 20%; End-Sem-30%, Lab Assignment-30%, Lab Attendance-5%		
Course Materials/ References	1. An Introduction to Parallel Programming; Elsevier; by Peter S. Pacheco. 2. Scientific Parallel Computing; Princeton University Press; by Babak Bagheri Terry Clark L Ridgway Scott Bagheri Clark Scott 3. PARALLEL PROGRAMMING; Barry Wilkinson, Pearson Education. 4. Introduction to High Performance Computing for Scientists and Engineers; G. Hager & G. Wellein. CRC Press. 5. Algorithms Sequential & Parallel: A Unified Approach, by Millers Russ; Cengage, ISBN 9788131525050		

	6. Parallel Programming in C with MPI and OpenMP; by Michael J. Quinn; McGraw-Hill Higher Education 7. Parallel Computing Theory and Practice. By Michael J. Quinn; McGraw Hill Education (India). 8. *Let's HPC*: A web-based platform to aid parallel, distributed and high performance computing education (https://doi.org/10.1016/j.jpdc.2018.03.001)
Detailed Course Content	**APPENDIX** **Instructor: Prof. Bhaskar Chaudhury (bhaskar_chaudhury@dau.ac.in)**

Course Outcome:
CO1- Ability to design and implement of parallel algorithms on shared and distributed memory architectures.
CO2 - Code optimization, profiling and performance analysis of parallel codes.
CO3 - Apply programming (OpenMP and MPI) skills in problem solving techniques.
Work in a group for laboratory assignment, and present their results

POs-COs Matrix:

P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12
x	x	x	x	x				x	x		x

APPENDIX: Detailed Course Content (Module-wise)

Course content [42 Lecture sessions]

Module 1: Overview of latest parallel machines and architecture. Introduction to high performance computing. Parallel Programming concepts. Need for Parallel Computing. Limitations. Pipelining, SIMD, Memory hierarchies, caches, Improving performance on a single processor: basic optimization techniques for serial code. Data access optimization, Measuring performance, bandwidth measures, thread vs process, thread synchronization, Memory structures and bandwidth optimization, vector triad benchmark, performance analyzer tools, debugging.

Module 2: Basics of parallelization. Parallel-serial problem breakdown, Data parallelism, Dependence analysis, Amdahl's law, Gustafson's Law, Karp–Flatt metric, isoefficiency metric.

Module 3: Multi-threading model using OpenMP. Data scoping, worksharing, synchronization, loop scheduling, OpenMP: parallel do, private variables, nested loops, reductions, loop dependencies, thread-safe functions, parallel sections, and barriers. Profiling OpenMP programs, Case-studies.

Module 4: Message Passing Programming, its implementation and important details, MPI send and receive, MPI communicators, broadcast, reduce. Blocking and non-blocking communication. Case-Studies

Module 5: Parallel complexity analysis, speedup, efficiency, work, span, cost, task and data dependency graphs. Scalability.

Case Studies to be covered in the above modules: Several Important Parallel Algorithms and implementation strategies from different class of problems such as Integration using trapezoidal rule. Vector addition, Calculation of PI using monte carlo method. Matrix operations. Reduction, Inclusive and exclusive scan. Interpolation. Sorting algorithms. Domain decomposition. Image processing. Solution of Differential Eqns using Finite Difference etc. Hybrid parallelization with MPI and OpenMP.